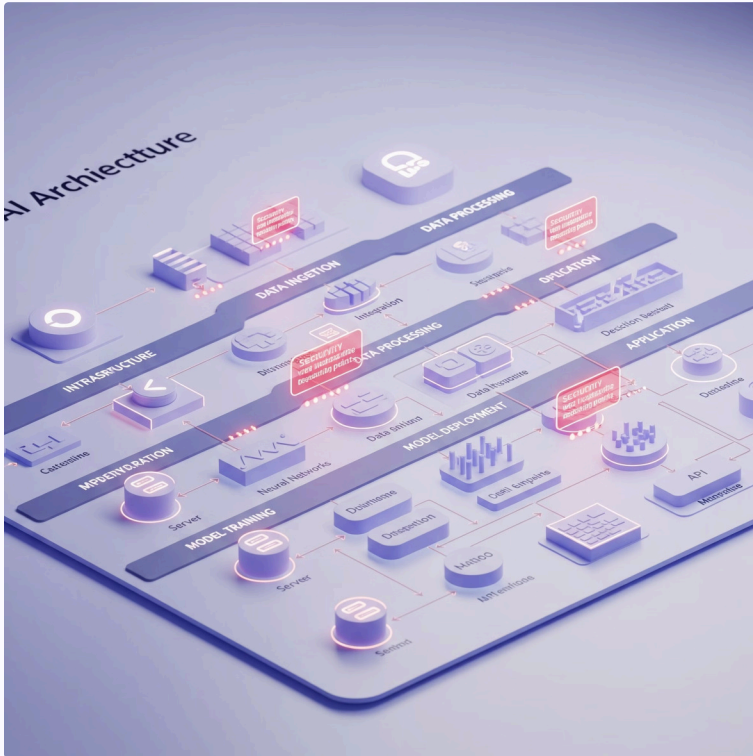




Secrets Management for AI Systems

Protecting keys, tokens, and credentials in ML pipelines

Why Secrets Management Matters



AI stacks use **many credentials** across multiple systems:

- Cloud API keys (AWS, GCP, Azure)
- Database passwords & feature store tokens
- Model registry & vector DB authentication



Hardcoding secrets leads to security breaches, compliance violations, and potential business-ending data exposures

Types of Secrets in AI Workflows



Data Access

S3/GCS/Blob storage credentials, SQL database passwords, NoSQL connection strings, data warehouse access tokens



Model Access

Hugging Face tokens, private model registry credentials, OpenAI API keys, vector embedding service authentication



Pipeline

CI/CD tokens, container registry credentials, artifact repository access, code signing certificates



Infrastructure

GPU cluster kubeconfigs, cloud IAM keys, provisioning credentials, monitoring system tokens



Common Pitfalls

Repository Leaks

API keys and tokens directly committed to GitHub repositories, even in private repos. Once exposed, credentials can be scraped within minutes by automated bots.

Static Credentials

Long-lived static keys without rotation policies. The average time to discovery for leaked AWS keys is less than 24 hours, but many organizations use the same keys for years.

Shared Accounts

Shared service accounts with no individual traceability. When everyone uses the same credentials, attribution becomes impossible during security incidents.

Environment Exposure

Environment variables exposed in container logs, debugging output, or error messages. ML system logs are particularly verbose and often capture entire environments.

Secrets Vaults & Managers

HashiCorp Vault

- Dynamic secrets with auto-expiry
- Policy-based access control
- Kubernetes integration via sidecar

AWS Secrets Manager

- Integrates with IAM roles
- Automatic RDS credential rotation
- SDK integration with Lambda

GCP Secret Manager

- Per-project secrets with versioning
- Built-in rotation scheduling
- Vertex AI integration points

Azure Key Vault

- RBAC + comprehensive audit trails
- Managed HSM backing
- AML workspace integration



Pro tip: Choose a secrets manager that integrates well with your existing cloud infrastructure and CI/CD pipelines.

Kubernetes Secrets



For ML workloads running on Kubernetes:

- Store small secrets as Kubernetes Secrets objects
- Enable **encryption at rest** with KMS backing
- Implement strict RBAC policies for access
- Mount as files or inject as environment variables (with caution)

⚠ Native K8s secrets have limitations. For production AI systems, prefer **external secret managers** via CSI drivers or the External Secrets Operator.

Example: Mount HuggingFace tokens as volume for model-serving containers without exposing in pod specs.



Secret Rotation & Expiry

Automated Rotation

Schedule regular key rotation at intervals of days or weeks, not years.
Critical ML pipeline credentials should rotate every 30-90 days.

1

2

Short-lived Tokens

For CI/CD pipelines training or deploying models, use tokens valid only for the duration of the job (minutes to hours).

Dynamic Secrets

With Vault or similar systems, generate credentials on demand with automatic revocation after use. Especially useful for one-time training jobs.

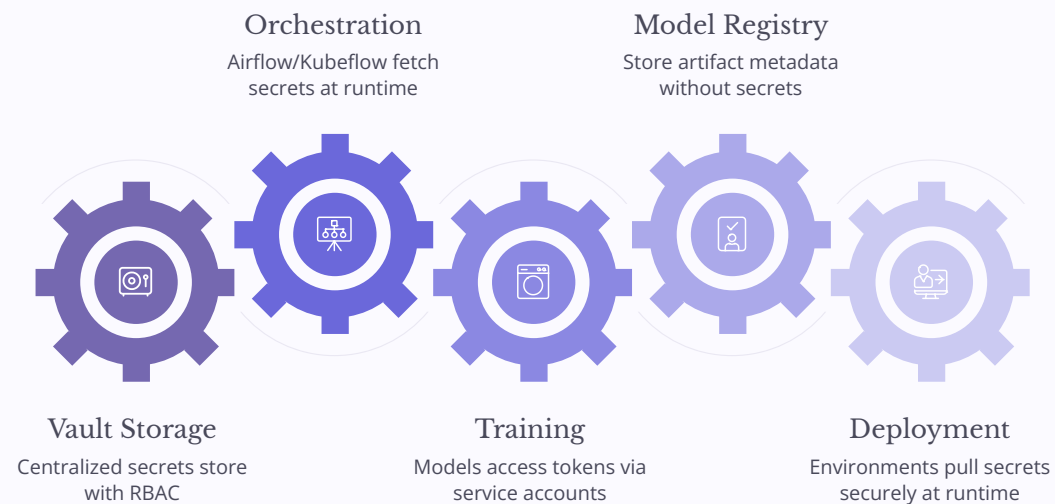
3

4

Immediate Expiry

When models or services are decommissioned, immediately expire their associated keys rather than waiting for rotation cycles.

Integration with AI Pipelines



Orchestration Integration

MLflow / Kubeflow / Airflow → fetch secrets at runtime from centralized vaults using service accounts

Container Security

Inject secrets at build time via secure mechanisms, never bake into images

Model Access

Hugging Face / OpenAI tokens → pulled from secure storage, never stored in .env files or notebooks

Deployment Security

Pull secrets only in specific pod namespaces with strict isolation

Monitoring & Auditing Secrets

Anomaly Detection

Configure alerts for unusual secret access patterns, such as retrievals from new locations or at unusual hours

Leak Detection

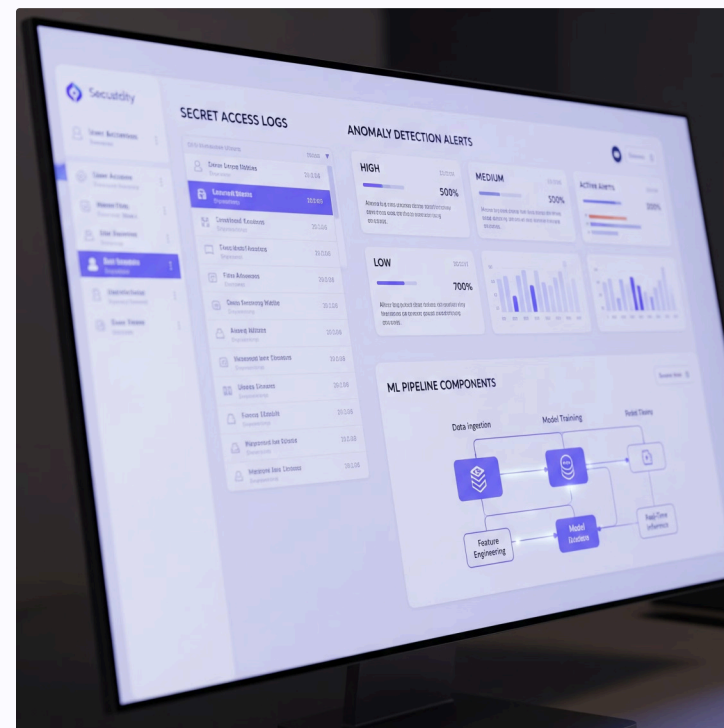
Run automated scanners (Trufflehog, GitLeaks) against repositories, logs, and model artifacts to find exposed credentials

Audit Trails

Maintain comprehensive logs of who accessed what secrets and when, with retention policies aligned to compliance requirements

Policy Enforcement

Implement automated checks to prevent secrets from appearing in code reviews, commits, or public notebooks



Did you know? 63% of organizations experienced security incidents related to secrets mismanagement in their ML pipelines last year.

Best Practices

1 No Hardcoding

Never hardcode secrets in code, notebooks, or configuration files. This includes "temporary" development keys that often become permanent.

2 Centralize Management

Use a dedicated secrets manager (Vault/AWS SM/Key Vault) as the single source of truth for all credentials across your AI infrastructure.

3 Automate Lifecycle

Implement automated rotation, expiry, and revocation for all credentials. Manual rotation is error-prone and often forgotten.

4 Least Privilege

Follow the principle that one secret should serve one specific purpose with minimal permissions required for that task.

5 Team Training

Regularly train ML engineers and data scientists on secrets hygiene in AI workflows, focusing on practical, secure alternatives to common shortcuts.

Key Takeaways



Foundation of Security

Secrets form the backbone of data & model security in AI systems



Complex Dependency Web

AI systems depend on many hidden credentials across the stack



Proactive Protection

Use vaults, rotation, and auditing to stay ahead of threats

Protecting your secrets is protecting your entire AI infrastructure.

Implement a secrets strategy before your first model goes to production—retrofitting security is much harder than building it in from the start.

Questions? Contact the **Security Team** for guidance on implementing these practices in your ML workflows.